

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

Search the Help Center 

[Fireblocks Help Center](#)

[Tokenization](#)

Tokenization best practices

Blockchain Bond Tokenization Best Practices

Overview

Blockchain provides an excellent solution for bond tokenization by enabling automation, transparency, and immutability. An Ethereum-based bond tokenization process usually involves an Ethereum Virtual Machine (EVM), smart contracts, and ERC-20 or ERC-1155 tokens.

Fireblocks Custody Services

Fireblocks provides the MPC-CMP private key creation and multi-layered protective storage. The EVM blockchain is defined in the Fireblocks system, which means that the transactions on that blockchain can be signed with the private key securely stored on the Fireblocks platform.

The Fireblocks Workspace

The Fireblocks *workspace* is a secure environment that allows users to manage digital assets, transactions, and other features based on their assigned user roles. It is a part of the Fireblocks platform architecture and can be either a Mainnet workspace or a Testnet workspace.

Fireblocks creates a specific workspace with the customer as a workspace Owner. The workspace *Owner* is the root user of a workspace and is responsible for approving Multi-Party Computation (MPC) signing devices and new users, in addition to all other Admin privileges.

The Owner can link multiple users to a workspace, and depending on their role, they can manage users, connect to counterparties, exchanges, and other external participants,

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help C

/

scenarios. The *vault* refers to a secure digital storage solution for storing digital assets such as cryptocurrencies or digital tokens. Vaults are designed to be highly secure and are protected by multiple layers of security, including MPC technology and other advanced security measures. Vaults can be created and managed through the Fireblocks Console and can be used to store a wide range of digital assets, including Bitcoin, Ethereum, and many others.

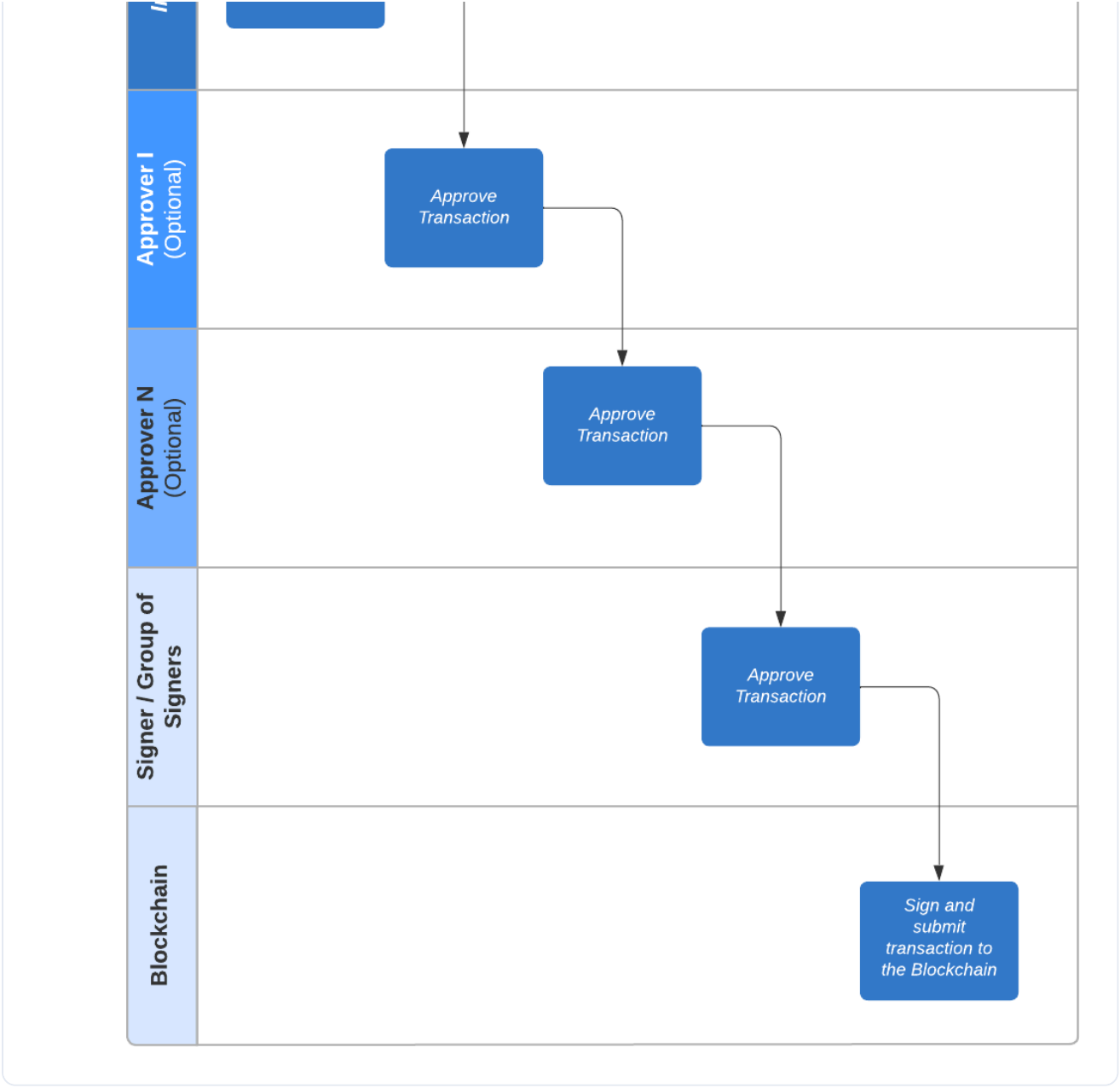
In the current implementation, the vaults contain wallets with “CBDC” and “Bond” tokens. The wallet is a BIP32-compatible Hierarchical Deterministic (HD) wallet and is used to store and manage cryptocurrencies. Fireblocks Vault is an example of a wallet that uses HD derivation paths to generate and manage wallet addresses. Wallets can be connected to Web3 decentralized applications (dApps) using protocols such as WalletConnect, and transactions made through these dApps are processed through the Fireblocks Vault for approval and MPC signing.

Users and User Roles

All users in a Fireblocks workspace require an assigned role. *User roles* grant permissions regarding which parts of the platform users can access, what type of actions they can perform, and whether they have MPC key shares used to sign transactions.

By combining different user roles, we create a process in which different entities can initiate, approve, and finally sign the transaction, therefore maximizing the security of the transaction approval process.

Don't see what you're looking for? Log in for the full Help Center experience



Blockchain Processes for EVM-based Bond Tokenization

Standard Smart Contracts

Standard smart contracts used in the processes below might include:

- **Bond contracts:** These represent individual bonds or batches of bonds, containing all the relevant details about the bond and handling the sale, coupon payments, and

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

- **Oracle contracts:** These might be used to provide data to the bond contracts, such as the current time (to determine when coupon payments are due), or the current exchange rate (if payments are made in a different currency).

Contract Deployment

The deployment of a smart contract refers to the process of writing a contract in a specific language (usually Solidity for the Ethereum blockchain), testing it, and then sending it to the blockchain. Once a smart contract is deployed, it cannot be changed.

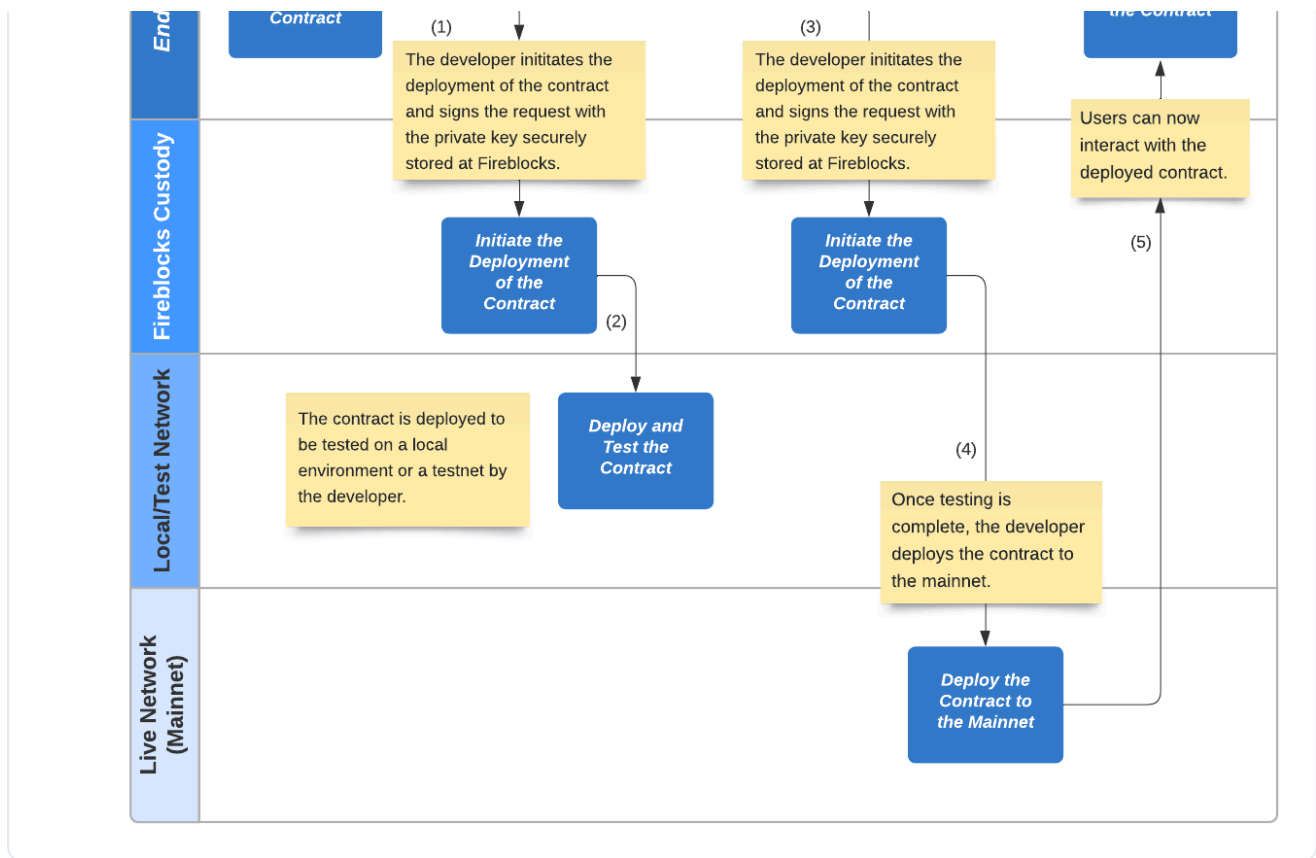
Here are the general steps involved in the smart contract deployment process:

1. **Smart Contract Development:** A developer writes the smart contract using a programming language that's compatible with the blockchain. For Ethereum, this is typically Solidity.
2. **Testing:** The contract is tested thoroughly to ensure it functions as expected. This might be done in a local development environment or on a testnet, which is a separate blockchain network used specifically for testing purposes.
3. **Deployment:** Once testing is complete, the contract is deployed to the Mainnet. This is done by sending a special transaction that includes the contract's code and is signed with the private key securely stored on Fireblocks.
4. **Interaction:** After the contract is deployed, users can interact with it by sending transactions that call its functions.

Don't see what you're looking for? Log in for the full Help Center experience

Fireblocks Fireblocks Help Center

/



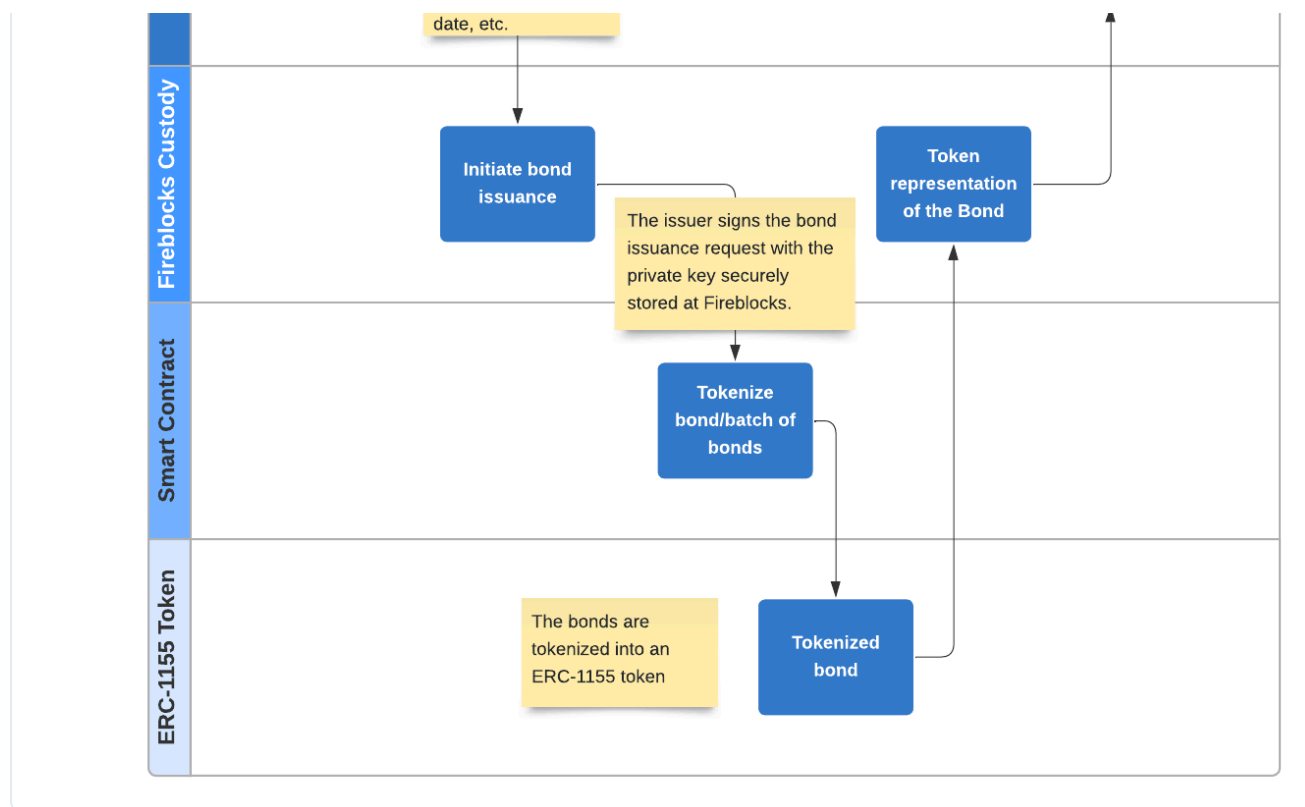
Bond Issuance

The issuer (e.g., a corporation or government) initiates the bond issuance process through a smart contract. The smart contract represents the bond terms, such as its par value, coupon rate, maturity date, and so on. Each bond or batch of bonds is then tokenized into an ERC-1155 token. The bond issuer's request is signed with the private key securely stored on Fireblocks.

Don't see what you're looking for? Log in for the full Help Center experience

Fireblocks Fireblocks Help Center

/



Bond Sale

Investors purchase the bond tokens by sending funds (e.g., Ether, a stablecoin, CBDC) to the smart contract. The smart contract verifies the payment and transfers the corresponding bond tokens to the investor's wallet.

Here are the general steps involved in the bond sale process:

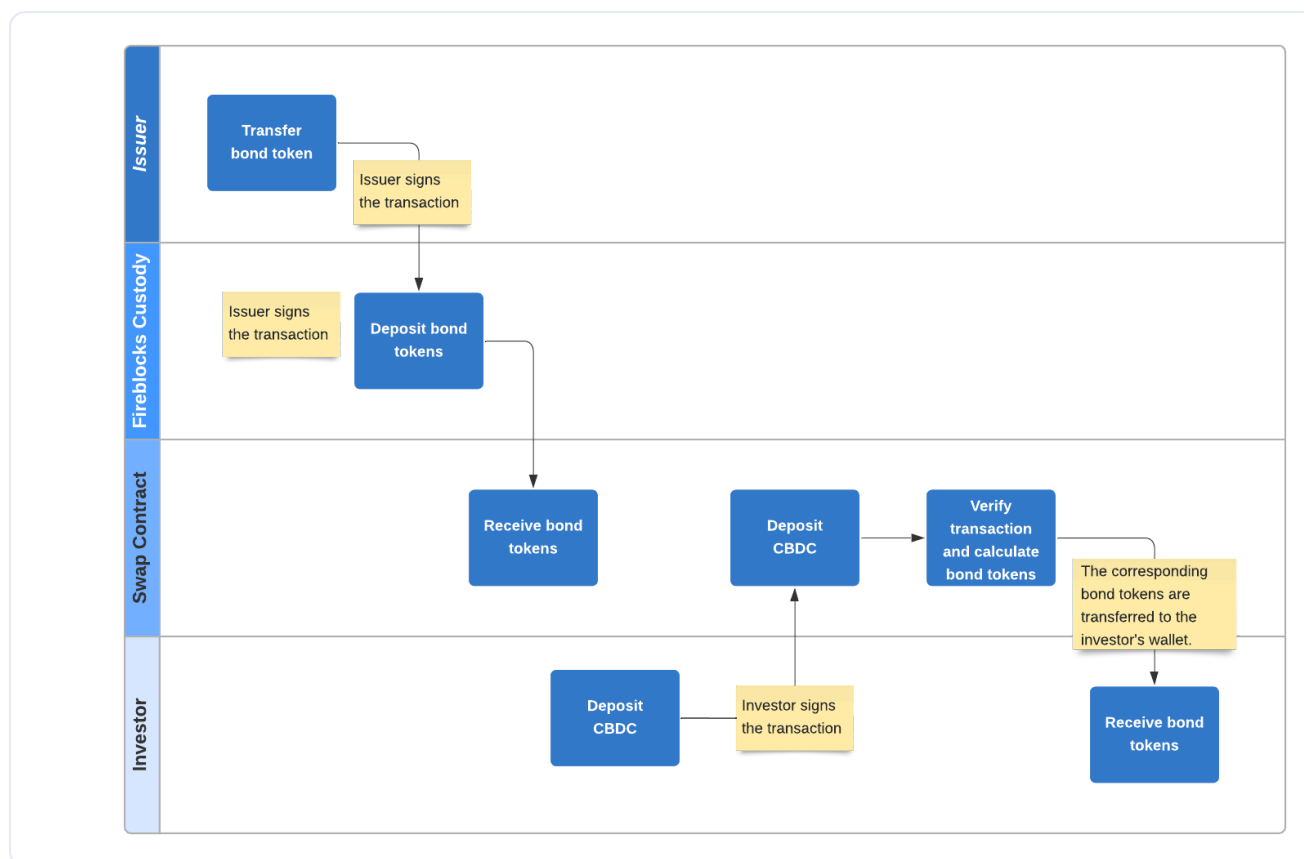
1. **Deployment:** First, the swap contract needs to be deployed on the blockchain. This usually involves coding the exchange rules into a smart contract and deploying it to the blockchain. For example, the rules could include the exchange rate between the bond tokens and the CBDC.
2. **Bond Deposit:** The issuer would then deposit the bond tokens into the swap contract. This could involve calling a function in the smart contract and sending the bond tokens to the address of the smart contract.
3. **Investor Swap:** When an investor wants to purchase bond tokens, they will send the required amount of CBDC to the swap contract. The swap contract would verify the transaction, update its internal state, and send the corresponding bond token amount to the investor's address.

Don't see what you're looking for? Log in for the full Help Center experience

Fireblocks Fireblocks Help Center

/

Sign in with the private key securely stored on hardware



Coupon Payments

Coupon payments are the periodic interest payments made to bondholders. In a traditional bond issuance, these payments are typically handled by a central entity, such as a bank or the issuer. When a bond is tokenized and managed by a smart contract, however, the process can be automated.

Periodically, the smart contract calculates the coupon payment for each bond token and sends it to the token holders. This can be set up to occur automatically at specified times, or it could require an action (e.g., a function call) to trigger the payment.

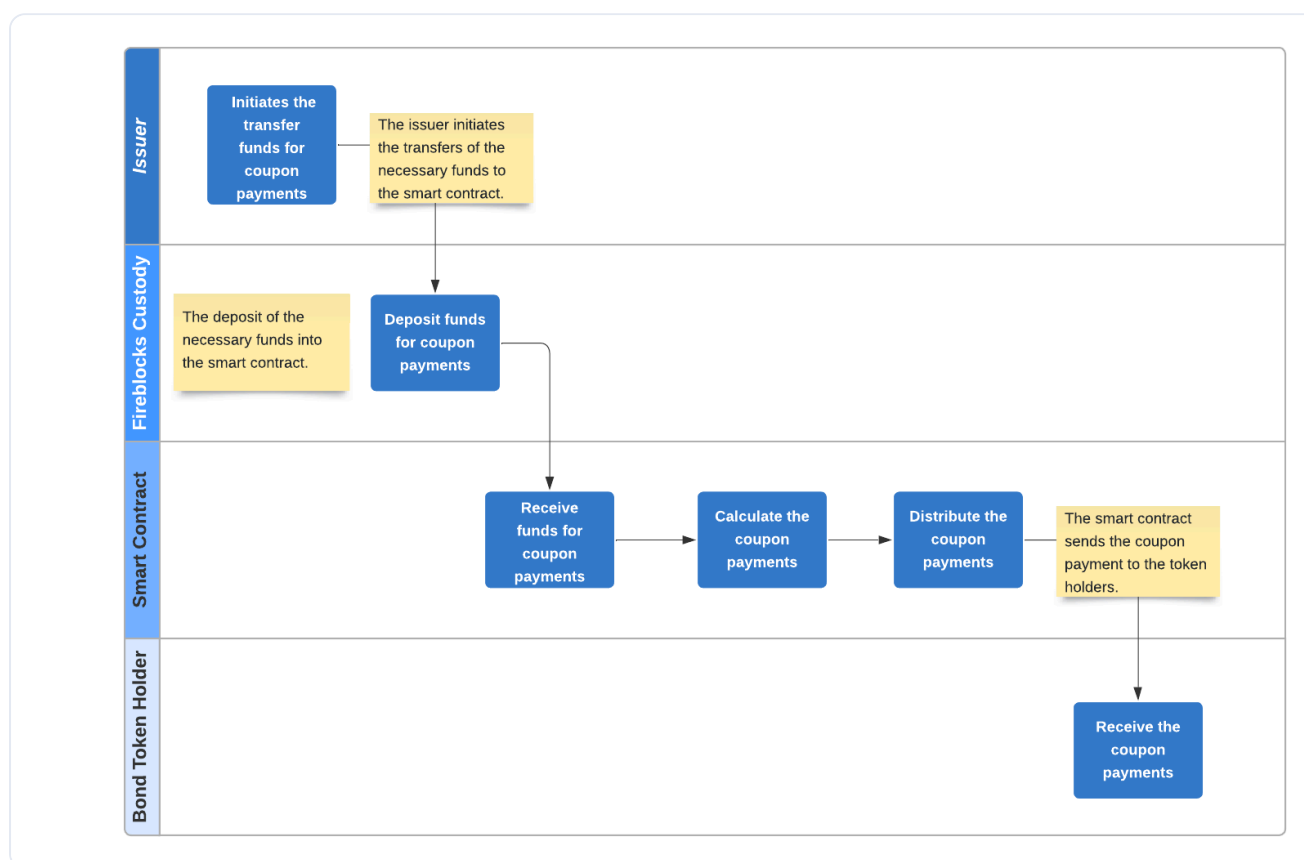
The funds used for coupon payments typically come from the issuer, who is responsible for paying the interest on the bonds. In the case of a tokenized bond managed by a smart contract, the issuer, or a custodian acting on behalf of the issuer, would have to deposit the funds needed for the coupon payments into the smart contract. The smart contract would hold these funds until the time to distribute them to the bondholders.

Don't see what you're looking for? Log in for the full Help Center experience

Fireblocks Fireblocks Help Center

/

- **Automatic Distribution:** The smart contract could be coded to distribute the coupon payments at certain times automatically. For example, if the bond has a semi-annual coupon, the smart contract could be programmed to distribute the payments every six months.
- **Triggered Distribution:** Alternatively, the distribution of coupon payments could require a specific function to trigger it. This could be done by the issuer, a custodian, or potentially even the bondholders themselves.



Bond Redemption

When the bond reaches its maturity date, the smart contract enables bondholders to redeem their bonds. The holders send their bond tokens to the smart contract, and the smart contract sends them the face value of the bonds.

When a bond reaches its maturity date, the issuer has to pay back the face value of the bond to the bondholder. In the case of a tokenized bond managed by a smart contract, this redemption process can be automated.

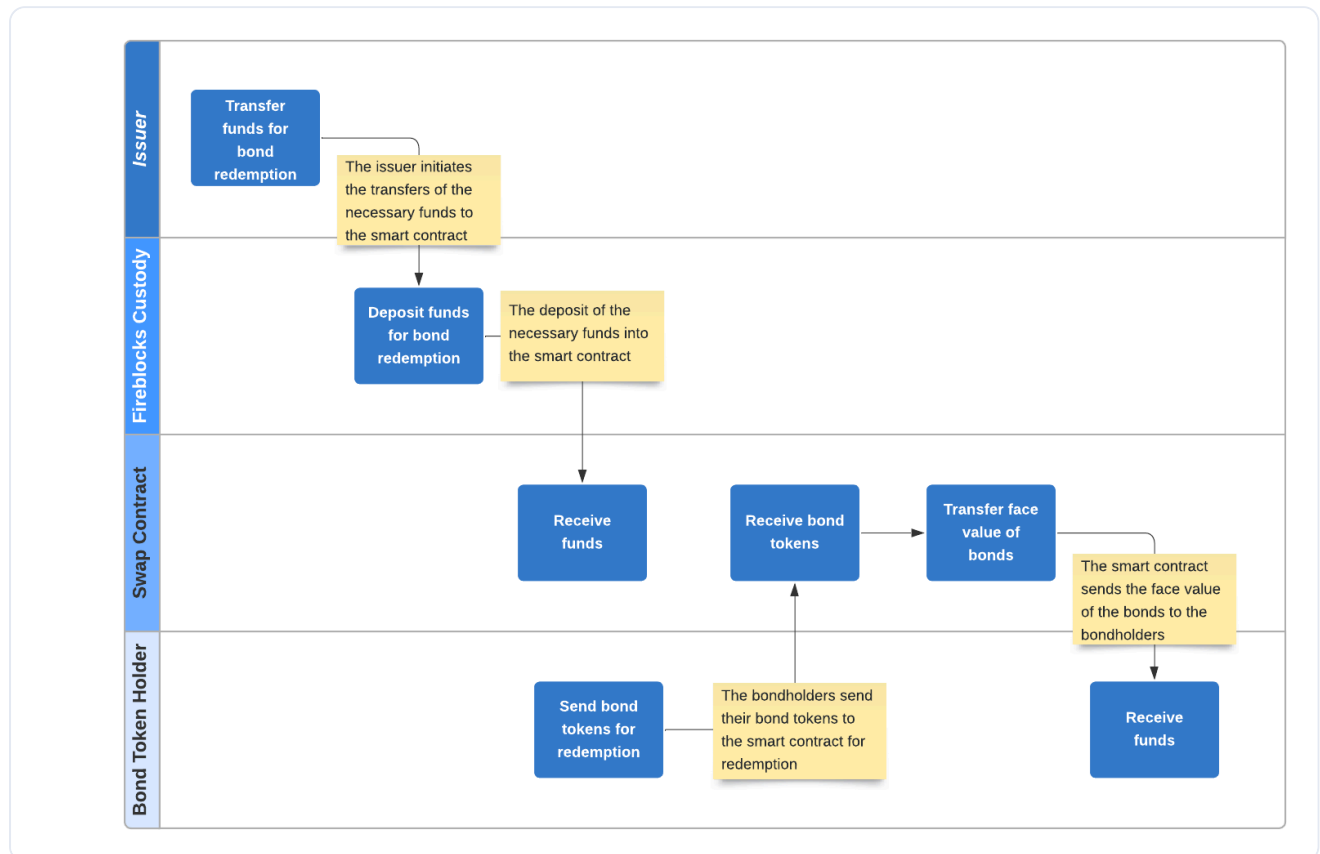
Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

Issuance and Redemption

The funds could be in the form of a CBDC or any other form of tokenized value. This would depend on the specific terms of the bond and the capabilities of the blockchain platform being used.



Was this article helpful?

Yes

No

ON THIS PAGE

Overview

Fireblocks Custody Services

Blockchain Processes for EVM-based Bond Tokenization

Contract Deployment

Bond Issuance

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

Related Articles

-  [Near tokens initial deposit](#)
-  [Tokenization on Ripple](#)
-  [Tokenization on Stellar](#)
-  [About tokenization on Fireblocks](#)
-  [Why does my token contract show a different address for roles?](#)
-  [Issuing tokens](#)
-  [Minting and burning tokens](#)
-  [Tokenization on EVM-compatible blockchains](#)



fireblocks.com

[Status](#)

[API Docs](#)

[Console](#)

info@fireblocks.com



Fireblocks © 2026. All Rights Reserved. NMLS Registration Number: 2066055

[Privacy Policy](#)